Appello del 18 Luglio 2011



Politecnico di Milano

Anno accademico 2010-2011

Ingegneria del Software

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 1

Si consideri la seguente specifica di una classe Lista. Una Lista e' una sequenza di elementi che consente ricerca, inserimento e eliminazione.

```
public class Lista<T extends Comparable<T>> {
/*OVERVIEW: Collezione mutabile di oggetti, di tipo T, organizzati in una sequenza.
 * Oggetto tipico di dimensione n: x1 x2 ... xn, in cui x1 e' il primo elemento.
 * Rimozione e inserimento di elementi possono avvenire in una posizione qualunque.
*/

//@ensures (*inizializza this alla  Lista vuota*);
public Lista()

//osservatori puri:

//@ensures (* \result == numero di elementi di this *)
public /*@ pure @*/ int size()

//@ensures (* \result <==> c'e' elemento equals(x) in this *)
public /*@ pure @*/ boolean  contains(T x)

//@requires i>=0 && i < size();
//@ensures (*  \result == i+1-esimo elemento di this, cosi' come sarebbe restituito
//@ dall'iteratore elementi(), ossia get(0) e' il primo elemento, ecc. *);
public /*@ pure @*/ T get(int i )

//mutators:

//@requires size()>0;
//@ ensures (*  elimina e restituisce il massimo elemento
//@ (secondo compareTo) in this *);
public T extractMax()


//@ensures (* aggiunge x in una qualunque posizione della lista *);
public void insert(T x)


//@requires size()>0;
//@ ensures (* elimina da this  una comparsa di un elemento equals(x) *)
public void remove(T x)
}

//@requires (*this non e' modificato mentre l'iteratore e' attivo*);
//@ensures (* \result e' un iteratore che itera, una sola volta, su ciascun elemento
//@ L'ordine degli elementi puo' cambiare solo se cambia la lista*);
public Iterator<T> elementi()
```

dove si rammenta la definizione dell'interfaccia Comparable:

```
public interface Comparable<T> {
  //@ensures (* if this<arg then \result ==-1
  //@  else if this == arg then \results == 0,
```

```
  //@     else \result == 1 *);
  public int compareTo(T arg);
}
```

Si noti che l'iteratore elementi(), se richiamato piu' volte sulla stessa Lista non modificata, restituisce gli elementi sempre nello stesso ordine (ossia, e' consistente).

Domanda A

Specificare in JML le postcondizioni dei metodi insert, extractMax, remove.

Soluzione: Uno dei vari modi per scrivera le postcondizioni e' il seguente.

insert:

```
/*@ensures     size() == \old(size())+1 &&
@               (\forall T y; \old(contains(y));
@                  (\num_of T z; contains(z); y.equals(z)) ==
@                  (\num_of T w; \old(contains(w); y.equals(w))
@                  + (x.equals(y) ? 1 : 0)
@*/
```

remove:

```
/*@ensures     size() == \old(size()) - (\old(contains(x)) ?  1 : 0) &&
@               (\forall T y; \old(contains(y));
@                  (\num_of T z; contains(z); y.equals(z)) ==
@                  (\num_of T w; \old(contains(w); y.equals(w))
@                  - (x.equals(y) ? 1 : 0)
@*/
```

extractMax:

```
/*@ensures     size() == \old(size())-1 && \old(contains(\result)) &&
@               (\forall T y; \old(contains(y));
@                    \result.compareTo(y)>=0 &&
@                    (\num_of T z; contains(z); y.equals(z)) ==
@                    (\num_of T w; \old(contains(w); y.equals(w))
@                    + (\result.equals(y) ? 1 : 0)
@*/
```

Domanda B

Si consideri una classe ListaOrdinata, definita come Lista, ma con la proprieta' che gli elementi sono disposti nella lista in ordine crescente. La classe ha gli stessi metodi di Lista con la stessa specifica, salvo insert, che e' specificato come segue:

```
//@ensures (* aggiunge x in this in una posizione fra un elemento <=x
//e un elemento >=x (usando compareTo per il confronto*);
public void insert(T x)
```

e salvo il metodo iteratore elementi(), che restituisce un iteratore che itera sugli elementi di this in ordine *crescente*. La get(i) deve sempre rispettare lo stesso ordine di elementi().

1. **E' possibile definire ListaOrdinata come erede di Lista o viceversa rispettando il principio di sostituzione? Se la risposta alla precedente domanda e' si, definire *in JML* la specifica della classe erede.**
2. **Definire un invariante astratto per la classe che specifica l'ordinamento della lista.**

Soluzione:
Ovviamente l'unica possibilita' e' che ListaOrdinata erediti da Lista, in quanto le postcondizioni di ListaOrdinata.insert e di ListaOrdinata.elementi() sono piu' forti delle corrispondenti di Lista. L'invariante e la specifica di ListaOrdinata sono le seguenti:

```
public class ListaOrdinata extends Lista {
//@public invariant (\forall int i; 0\le && i<size()-1; get(i).compareTo(get(i+1)<=0);

//@also
//@ensures (* \result e' un iteratore che itera, una sola volta, su ciascun elemento di this i
public Iterator<T> elementi()

//@also
//@ensures (\exists int i;
              0\le i && i < size();
              get(i).equals(x) &&
              (i+1<size() ==> x.compareTo(get(i+1))>=0) &&
              (i-1\ge 0 ==> x.compareTo(get(i-1))<=0)
          );
public void insert(T x)
}
```

NB: se insert e' stata definita correttamente per Lista, l'inserimento dell'elemento e' gia' garantito, ed e' quindi sufficiente aggiungere la posizione corretta.

Domanda C

Si consideri una classe ListaStrana, definita come Lista, ma che memorizza gli elementi in un ordine "strano". Vale a dire, gli elementi della lista sono disposti in un ordine che puo' essere inizialmente crescente e poi decrescente (le due porzioni di lista crescente e decrescente possono essere nulle). La classe ListaStrana fornisce una operazione di inserimento analoga a quella definita in precedenza per ListaOrdinata, che inserisce un elemento nella porzione crescente di valori. Fornisce anche una operazione *insert1* che inserisce l'elemento nella parte decrescente di valori.

1. **Definire un invariante astratto per la classe ListaStrana.**
2. **E' possibile definire ListaStrana come erede di ListaOrdinata o viceversa rispettando il principio di sostituzione?**
3. **Definire le specifiche JML per insert e insert1.**

Soluzione: 1)

```
public invariant
size()>0 ==>
  (\exists int j; 0<=j && j<size();
    (forall int i; 0<=i && i<size();
      get(i) != null &&
      (i<j ? get(i).compareTo(get(i+1))<=0 : get(i).compareTo(get(i+1))>=0)));
```

Di fatto, la posizione j corrisponde al massimo elemento nella ListaStrana.

2) ListaOrdinata e' un caso speciale di ListaStrana, ma non puo' essere definita come sottoclasse che rispetti il principio di sostituzione in quanto essa eredita pure l'operazione insert1 che non si applica a ListaOrdinata. Anche il viceversa non e' possibile, in quanto ListaStrana violerebbe la proprieta' dell'ordinamento di ListaOrdinata (bastano due chiamate a insert1 per introdurre un "pezzo" decrescente).

3) insert contiene come postcondizione la stessa specifica di Lista.insert, ma inserisce solo nella porzione j, definita dalla stessa formula usata nell'invariante, che stabilisce l'ordinamento:

```
size() == \old(size())+1 && //questa e' come in Lista.insert
        (\forall T y; \old(contains(y));
             (\num_of T z; contains(z); y.equals(z)) ==
             (\num_of T w; \old(contains(w); y.equals(w))
             + (x.equals(y) ? 1 : 0) &&
\old(size()>0)==>(\exists int j; 0<=j && j<size(); //qui e' come in invariante
   (forall int i; 0<=i && i<size();
      (i<j ? get(i).compareTo(get(i+1))<=0 : get(i).compareTo(get(i+1))>=0)) &&
   (\exists int k; 0\le k && k <=j; get(k).equals(x)); //x sta in prima parte
```

La insert1 e' simmetrica.

Domanda D

Si consideri una classe ListaEstesa, definita come Lista, ma che definisce un altro ordine in cui si trovano gli elementi. A tale scopo, ListaEstesa fornisce, oltre alle operazioni di Lista, le seguenti operazioni:

```
//@ensures (* aggiunge x nella posizione immediatamente precedente al primo elemento
//@ equals(y). Se y non esiste in this, allora inserisce x in ultima posizione *);
public void insertAfter(T x, T y)
```

e ridefinisce elementi() in modo che restituisca tutti e soli gli elementi di this nell'ordine di *inserzione*.

1. **Specificare insertAfter in JML.**
2. **E' possibile definire ListaEstesa come erede di Lista?**
3. **Come cambia la risposta alla Domanda B(1) se al posto di Lista si considera ListaEstesa?**

Soluzione:

```
//@ensures ( (\exists int i; 0<i && i<size(); get(i).equals(y) && get(i-1).equals(x)  ||
     !(\exists int i; 0<i && i<size(); get(i).equals(y))  && get(size()-1).equals(x))
    && (* qui stessa parte che in insert assicura che gli elementi sono rimasti gli stessi sal
public void insertAfter(T x, T y)
```

ListaEstesa puo' essere erede di Lista, in quanto Lista non prevede particolare vincoli sull'inserimento.

ListaOrdinata non puo' ereditare da ListaEstesa (in quanto il metodo insertAfter sarebbe ereditato), ma nemmeno il viceversa, in quanto ListaEstesa viola la regola delle proprieta' consentendo tramite insertAfter di inserire in posizioni che non rispettano l'ordine.

Domanda E

Si consideri la seguente implementazione di ListaOrdinata (che si suppone definita "da zero", ossia senza estendere Lista o ListaEstesa o ListaStrana).

```
\\rep:
private ArrayList<T> elems;
private ArrayList<Integer> ordine;

public ListaOrdinata ()
  elems = new ArrayList<T>();
}
```

Il vettore *elems* contiene tutti gli elementi della lista, in un ordine qualunque. Il vettore *ordine* contiene per ogni posizione i, $0 \leq i \leq elems.size() - 1$, la posizione del successore in elems di $elems.get(i)$. Per l'ordinamento, si usa sempre il compareTo. Se l'elemento in posizione i non ha un successore (ossia e' il max), *ordine* in posizione i contiene -1.

Ad esempio, se elems contiene [A E B F C], allora ordine contiene [2 3 4 -1 1]

Scrivere l'invariante di rappresentazione in JML per ListaOrdinata.

Soluzione:

```
private invariant elems.size() == ordine.size() &&
  (\num_of int j; 0<= j && j <elems.size(); elems.get(j) ==-1) ==1 &&
  (\forall int i; 0<= i && i <elems.size();
      elems.get(i)!=null && ordine.get(i)!=null &&
      (ordine.get(i)!=-1 ? ordine.get(i) >=0 &&
                           ordine.get(i) < ordine().size() &&
                           elems.get(i).compareto(elems.get(ordine.get(i)))<=0 &&
                           !(exists int j; j>=0 && j<elems.size();
                              j!= i && elems.get(i).compareto(elems.get(j)<0 &&
                              elems.get(j).compareTo(elems.ordine(get(i)))>0))
                         : (forall int j; 0<=j && j<elems.size();
                              elems.get(i).compareTo(elems.get(j))>=0)) &&
```

Usando la notazione preferita (tramite oggetto tipico, oppure un private invariant in JML, oppure implementando il metodo toString() di ListaOrdinata in Java), **descrivere la funzione di astrazione della ListaOrdinata**.

Soluzione: Esprimiamo la funzione di astrazione ad esempio con un invariante che mette in relazione i metodi osservatori puri get() e size() con il rep.

```
private invariant
  size()== elems.size() &&
  get(size()-1) == elems(ordine.indexof(-1)) &&
  (\forall int i; 0<= i && i<size()-1; get(i) == ordine.indexof(i)-1);
```

Implementare il metodo insert() di ListaOrdinata. **Soluzione:**

```
elems.add(x); //aggiunge x come ultimo elemento
if (elems.size()== 1) {
    ordine.add(-1);
    return;
}
int minimo = 0; //indice del minimo elemento (da trovare)-- all'inizio e' 0;
(for int i=0; i<ordine.size(); ++i) {
if (elems.get(i).compareTo(elems.get(minimo))<0)
    minimo =i; //elems.get(i) e' il minimo dei primi i elementi
        int succ = ordine.get(i); //succ contiene indice del successore di elems.get(i);
        if (succ == -1 ) //se elems.get(i) e' il massimo elemento
```

```
    if (elems.get(i).compareTo(x) == -1) {//se il massimo e' < x
      ordine.add(-1); //il nuovo max e' x, ossia l'ultimo elemento
      ordine(i)= elems.size()-1; //il succ di i e' proprio x, in ultima posiz.
      return;
    }
else {
  if (elems.get(i).compareTo(x)<0 && elems.get(succ).compareTo(x)>0) {
    //se x e' compreso fra elems.get(i) e il suo successore:
    ordine.add(succ); //x ha come successore il succ. di elems.get(i)
    ordine.set(i,elems.size()-1); //elems.get(i) ha x come succ.
    return;
  }
}
//in uscita dal ciclo, x deve essere minore del minimo elemento;
ordine.add(minimo);
```

Domanda F

Dato il rep definito alla Domanda E, si definisca e implementi il metodo iteratore elementi() di ListaOrdinata

Soluzione:

```
public Iterator<T> elementi() {return new elgen<T>(this);}

private static class elgen<T> implements Iterator<T> {
  private int current;
  private ListaOrdinata<T> lista;
  public elgen<T>(ListaOrdinata c) {
    lista = c;
    if (c.size() == 0) {//lista vuota
      current = -1;
      return;
    }
    current =0;//ora cerca il minimo elemento
    (for int i = 1; i<lista.ordine.size(); i++)
      if (elems.get(current).compareTo(elems.get(i))>0) current = i;
    //in uscita, current punta al minimo elemento di elems
  }
  public boolean hasNext() {
    return current >=0;
  }
  public T next() throws NoSuchElementException {
    if (current<0) throw new NoSuchElementException();
    T temp = elems.get(current);
    current = ordine.get(current);
//quando current e' -1, temp e' il massimo, ossia l'ultimo elem. da restituire
    return temp;
   }
}
```

Esercizio 2

Si consideri il seguente frammento di programma

```
1. ``legge n da input'';
2. if (n>0) {
3.            j = 2*n;
4.            while (j>0) {
                    ----//qui codice che non cambia n e j
5.                  j--;
6.            }
7  ----;
8. };
```

1. Si definiscano casi di test che coprano tutte le istruzioni (si considerino le parte tralasciate "—-" come se fossero una singola macro-istruzione);
2. Si definiscano casi di test che coprono tutti i branch.
3. Si consideri il cammino: 1 2 3 4 5 6 4 7 (che attraversa il ciclo 1 sola volta) e si derivi sia una condizione logica sull'input n, che causi l'attraversamento del cammino, che valori specifici di input che ne causano l'attraversamento.
4. Si risolva lo stesso problema per il cammino 1 2 3 4 5 6 4 5 6 4 7 (che corrisponde ad eseguire il ciclo due volte)

Soluzione: 1) Basta prendere $n = 1$.
2) Basta prendere $n = 0$ e $n = 1$.
3) La condizione e' $0 < n \leq 0.5$. Se n e' un int allora non esiste alcun valore, se fosse un float basterebbe ad es. $n = 0.5$.
4) La condizione e' $0.5 < n \leq 1$, ossia basta prendere $n = 1$.